1. Introduction to DBMS

A **Database Management System (DBMS)** is software that allows users to define, create, maintain and control access to a database. A **database** is an organised collection of logically related data, designed to meet the information needs of an organisation. Instead of every application storing its own data in private files, a DBMS keeps data in one central place and provides a controlled interface through which many users and programs can work with it at the same time.

Databases are everywhere. A bank stores accounts and transactions, a university stores students, courses and grades, an online shop stores products, customers and orders, and a hospital stores patients, doctors and prescriptions. In each case the DBMS is responsible for storing the data efficiently, retrieving it quickly, keeping it consistent and protecting it from failures and unauthorised access.

Core functions of a DBMS

- **Data definition** - describing the structure of data using a Data Definition Language (DDL).
- **Data manipulation** - inserting, updating, deleting and querying data using a Data Manipulation Language (DML).
- **Data security** - ensuring that only authorised users can see or change particular data.
- **Integrity** - enforcing rules such as 'salary must be positive' or 'every order must belong to a customer'.
- **Concurrency control** - letting many users work simultaneously without corrupting data.
- **Backup and recovery** - restoring the database to a correct state after crashes or errors.

Components of a database system

A complete database system consists of **hardware** (servers, disks, network), **software** (the DBMS itself, the operating system and application programs), **data** (both the operational data and the metadata that describes it), **procedures** (instructions on how to use and run the system) and **people**. The people involved include database administrators (DBAs) who manage the system, database designers who decide the structure, application developers who write programs and end users who query and update the data.

Popular DBMS products include Oracle Database, MySQL, PostgreSQL, Microsoft SQL Server, SQLite, IBM Db2, and NoSQL systems such as MongoDB, Cassandra and Redis.

2. File System vs DBMS

Before database systems became common, organisations stored data in ordinary operating-system files. Each application program defined its own files and its own format. For example, a payroll program and a personnel program might both store employee names and addresses, each in separate files. This **file-processing approach** works for small problems but causes serious difficulties as systems grow.

Problems with the file-processing approach

- **Data redundancy and inconsistency** - the same data is stored in several files; when one copy changes and another does not, the data becomes inconsistent.
- **Difficulty in accessing data** - every new kind of query requires a new program to be written.
- **Data isolation** - data is scattered in different files with different formats, making it hard to combine.
- **Integrity problems** - consistency rules are buried inside program code and are hard to change or enforce.
- **Atomicity problems** - if a system crashes in the middle of a funds transfer, money may be debited but never credited.
- **Concurrent-access anomalies** - two users updating the same file simultaneously can overwrite each other's changes.
- **Security problems** - it is difficult to give users access to only part of the data.

Comparison

Aspect	File System	DBMS
Redundancy	High, uncontrolled	Minimised and controlled
Consistency	Hard to maintain	Enforced via constraints
Querying	Custom programs	Declarative query language (SQL)
Concurrency	Little or no support	Locking / MVCC built in
Recovery	Manual backups	Automatic logging and recovery
Security	File-level permissions	Fine-grained, per table/column
Data independence	None	Logical and physical independence

A DBMS does have costs: the software and hardware can be expensive, it adds complexity, and a small application may run slightly slower because of the general-purpose overhead. For most real-world multi-user applications, however, the benefits far outweigh these drawbacks.

3. DBMS Architecture

The most widely accepted framework for describing a database system is the **ANSI/SPARC three-schema architecture**. Its goal is to separate the way users see data from the way the data is physically stored, so that each can change without affecting the other.

The three levels

- **External level (view level)** - describes the part of the database relevant to a particular user group. A student sees only their own grades; an accountant sees only fee data. There can be many external schemas.
- **Conceptual level (logical level)** - describes *what* data is stored for the whole community of users and the relationships among the data: entities, attributes, relationships and constraints, without storage details.
- **Internal level (physical level)** - describes *how* the data is actually stored: file organisation, record formats, indexes, compression and the placement of data on disk.

Data independence

Logical data independence is the ability to change the conceptual schema (for example, adding a new column or table) without having to rewrite external schemas or application programs. **Physical data independence** is the ability to change the internal schema (for example, adding an index or moving files to faster disks) without changing the conceptual schema. Physical independence is easier to achieve and is provided by almost all modern systems.

Two-tier and three-tier architectures

In a **two-tier (client-server)** architecture, the application runs on the client machine and communicates directly with the database server, often through ODBC or JDBC. In a **three-tier** architecture, a middle application server sits between the client (usually a web browser) and the database server. The application server contains the business logic, which improves security, scalability and maintainability. Most modern web applications follow the three-tier model.

Main components of a DBMS

Internally a DBMS consists of a **query processor** (DDL interpreter, DML compiler, query optimiser and evaluation engine) and a **storage manager** (authorisation and integrity manager, transaction manager, file manager and buffer manager). The **data dictionary** or system catalog stores metadata such as table definitions, constraints, users and privileges.

4. Data Models

A **data model** is a collection of concepts used to describe the structure of a database, the relationships among data, the meaning of the data and the constraints that apply to it. Data models give designers a common vocabulary and determine what kinds of operations are possible.

Categories of data models

- **High-level (conceptual) models** - close to how users perceive data, e.g. the Entity-Relationship model.
- **Representational (implementation) models** - understood by users but close to how data is organised in the computer, e.g. the relational model.
- **Low-level (physical) models** - describe storage details such as record formats and access paths.

Historical data models

The **hierarchical model** (e.g. IBM IMS, 1960s) organises data as a tree: each child record has exactly one parent. It is efficient for one-to-many relationships but awkward for many-to-many ones. The **network model** (CODASYL) generalises this to a graph, allowing a record to have several parents; it is flexible but programs must navigate pointer chains, which makes them complex.

The relational model

Proposed by E. F. Codd in 1970, the **relational model** represents all data as tables (relations). Relationships are expressed through shared values rather than physical pointers. Its mathematical foundation, simplicity and the declarative SQL language made it the dominant model from the 1980s onward.

Other models

- **Object-oriented model** - stores objects with attributes and methods, supporting inheritance and complex types.
- **Object-relational model** - extends relational systems with user-defined types, arrays and JSON (e.g. PostgreSQL).
- **Semi-structured model** - XML and JSON documents whose structure can vary from record to record.
- **NoSQL models** - key-value, document, column-family and graph models designed for scale and flexibility.

A **schema** is the description of a database according to a data model and changes rarely. An **instance** (or state) is the actual data stored at a given moment and changes frequently.

5. Entity-Relationship (ER) Model

The **Entity-Relationship model**, introduced by Peter Chen in 1976, is the most popular tool for conceptual database design. It describes the real world as a set of entities, their attributes and the relationships among them, and is drawn as an **ER diagram**.

Entities and attributes

An **entity** is a real-world object distinguishable from other objects, such as a particular student or course. An **entity set** is a collection of similar entities. Each entity is described by **attributes**:

- **Simple vs composite** - Age is simple; Name may be composite (first, middle, last).
- **Single-valued vs multivalued** - a person has one date of birth but may have several phone numbers.
- **Stored vs derived** - Age can be derived from the stored Date of Birth.
- **Key attribute** - uniquely identifies an entity, such as Roll Number.

Relationships and cardinality

A **relationship** is an association among entities, for example a student *enrolls in* a course. The **degree** is the number of participating entity sets (binary, ternary). **Cardinality ratios** specify how many entities can participate:

Cardinality	Example
One-to-one (1:1)	Each country has one capital city
One-to-many (1:N)	One department employs many employees
Many-to-one (N:1)	Many students belong to one hostel
Many-to-many (M:N)	Students enrol in many courses; courses have many students

Participation constraints may be *total* (every employee must work in a department, drawn as a double line) or *partial* (not every employee manages a department).

Weak entities and extended features

A **weak entity** cannot be identified by its own attributes alone and depends on an owner entity; for example, a Dependent is identified by the employee's ID plus the dependent's name. The **Extended ER (EER)** model adds **specialisation** and **generalisation** (Employee may be specialised into Engineer and Manager, with inheritance of attributes) and **aggregation**, which treats a relationship as an entity so it can take part in another relationship.

6. The Relational Model

In the relational model, a database is a collection of **relations**, informally called tables. Each relation has a name and a set of named **attributes** (columns). Each row is called a **tuple**. The set of permitted values for an attribute is its **domain**.

Terminology

Formal term	Informal term	Meaning
Relation	Table	A set of tuples with the same attributes
Tuple	Row / record	A single entry in the table
Attribute	Column / field	A named property of the relation
Degree	Number of columns	How many attributes the relation has
Cardinality	Number of rows	How many tuples are currently stored
Domain	Data type	Set of allowed values for an attribute

Properties of relations

- Each relation has a distinct name, and each attribute in a relation has a distinct name.
- Every cell contains a single atomic value (first normal form).
- All values of an attribute come from the same domain.
- The order of tuples and the order of attributes carry no meaning.
- There are no duplicate tuples, because a relation is a set.

Integrity constraints

Domain constraints require each value to belong to its attribute's domain. **Entity integrity** says the primary key of a relation can never be NULL. **Referential integrity** says a foreign key value must either match an existing primary key value in the referenced relation or be NULL. In addition, designers may define general **semantic constraints**, for example 'a student may not enrol in more than eight courses per semester'.

Example

```
STUDENT(roll_no, name, dept_id, cgpa)
DEPARTMENT(dept_id, dept_name, building)

STUDENT.dept_id  references  DEPARTMENT.dept_id
```

Here roll_no and dept_id are primary keys, and STUDENT.dept_id is a foreign key. Mapping an ER diagram to relations follows simple rules: each strong entity becomes a table, a 1:N relationship becomes a foreign key on the N side, and an M:N relationship becomes a separate table whose key combines the keys of both participants.

7. Keys in DBMS

Keys are attributes, or sets of attributes, that help uniquely identify tuples and establish relationships between tables. Understanding the different kinds of keys is essential for good design and for normalisation.

Types of keys

- **Super key** - any set of attributes that uniquely identifies a tuple. {roll_no}, {roll_no, name} and {email, phone} might all be super keys.
- **Candidate key** - a minimal super key: removing any attribute makes it no longer unique. A table may have several candidate keys, such as roll_no and email.
- **Primary key** - the candidate key chosen by the designer to identify tuples. It must be unique and NOT NULL.
- **Alternate key** - a candidate key that was not chosen as the primary key.
- **Composite key** - a key made of two or more attributes, such as (student_id, course_id) in an ENROLLMENT table.
- **Foreign key** - an attribute in one table that refers to the primary key of another table, linking the two.
- **Surrogate key** - an artificial key, usually an auto-incremented integer or UUID, with no business meaning.
- **Unique key** - enforces uniqueness like a primary key but usually allows a NULL value.

Example in SQL

```
CREATE TABLE department (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE employee (
    emp_id     INT PRIMARY KEY,
    email      VARCHAR(100) UNIQUE,          -- alternate key
    name       VARCHAR(60) NOT NULL,
    dept_id    INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
        ON DELETE SET NULL
);
```

Choosing a primary key

A good primary key should be unique, never NULL, stable (it should not change over time), and preferably short. Natural keys like national ID numbers carry real meaning but may change or raise privacy concerns, so many systems prefer surrogate keys. Foreign keys may specify **referential actions** such as CASCADE, SET NULL or RESTRICT to decide what happens when a referenced row is deleted or updated.

8. Relational Algebra

Relational algebra is a procedural query language that takes one or two relations as input and produces a new relation as output. It forms the theoretical foundation of SQL and is used internally by query optimisers to represent and rewrite queries.

Fundamental operations

Operation	Symbol	Description
Selection	σ	Selects rows satisfying a condition
Projection	π	Selects specified columns, removing duplicates
Union	∪	Tuples in either relation (union-compatible)
Set difference	−	Tuples in the first relation but not the second
Cartesian product	×	Every tuple of R combined with every tuple of S
Rename	ρ	Renames a relation or its attributes

Derived operations

- **Intersection** (∩) - tuples present in both relations.
- **Natural join** (⋈) - combines tuples with equal values on common attributes.
- **Theta join** - a Cartesian product followed by a selection with any condition.
- **Outer joins** - left, right and full outer joins keep unmatched tuples, padding with NULLs.
- **Division** (÷) - answers 'for all' queries, such as students who have taken every course.

Examples

Find names of students in the CSE department:

```
π name ( σ dept = 'CSE' (STUDENT) )
```

Find names of students together with the names of courses they are enrolled in:

```
π STUDENT.name, COURSE.title ( STUDENT ⋈ ENROLLMENT ⋈ COURSE )
```

Relational calculus is a non-procedural alternative: **tuple relational calculus** and **domain relational calculus** describe *what* to retrieve rather than *how*. Codd proved that relational algebra and safe relational calculus have the same expressive power; a language with this power is called **relationally complete**.

9. SQL Fundamentals

SQL (Structured Query Language) is the standard language for relational databases. Developed at IBM in the 1970s as SEQUEL, it was standardised by ANSI and ISO and is supported, with small dialect differences, by every major relational DBMS.

Sub-languages of SQL

Category	Purpose	Commands
DDL	Define structure	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Manipulate data	INSERT, UPDATE, DELETE, MERGE
DQL	Query data	SELECT
DCL	Control access	GRANT, REVOKE
TCL	Control transactions	COMMIT, ROLLBACK, SAVEPOINT

Creating and modifying tables

```
CREATE TABLE student (
    roll_no  INT PRIMARY KEY,
    name     VARCHAR(50) NOT NULL,
    dept     VARCHAR(10) DEFAULT 'CSE',
    cgpa     DECIMAL(3,2) CHECK (cgpa BETWEEN 0 AND 10)
);

ALTER TABLE student ADD email VARCHAR(100);
```

Inserting, updating and deleting

```
INSERT INTO student (roll_no, name, dept, cgpa)
VALUES (101, 'Asha', 'CSE', 8.75), (102, 'Ravi', 'ECE', 7.90);

UPDATE student SET cgpa = 8.10 WHERE roll_no = 102;

DELETE FROM student WHERE cgpa < 5.0;
```

Basic queries

```
SELECT name, cgpa
FROM   student
WHERE  dept = 'CSE' AND cgpa >= 8
ORDER  BY cgpa DESC;
```

Useful operators include LIKE for pattern matching ('A%' matches names starting with A), IN for membership, BETWEEN for ranges and IS NULL for missing values. DISTINCT removes duplicate rows from the result. Note that DELETE removes selected rows and can be rolled back, TRUNCATE quickly removes all rows, and DROP removes the table itself.

10. Advanced SQL Queries

Real applications rarely query a single table. SQL provides joins, aggregation, subqueries and window functions to answer complex questions declaratively.

Joins

```
SELECT s.name, c.title
FROM   student s
JOIN   enrollment e ON e.roll_no = s.roll_no
JOIN   course c     ON c.course_id = e.course_id;
```

- **INNER JOIN** returns only matching rows from both tables.
- **LEFT / RIGHT OUTER JOIN** also returns unmatched rows from the left or right table.
- **FULL OUTER JOIN** returns unmatched rows from both sides.
- **CROSS JOIN** returns the Cartesian product; a **SELF JOIN** joins a table to itself.

Aggregation and grouping

```
SELECT   dept, COUNT(*) AS students, AVG(cgpa) AS avg_cgpa
FROM     student
GROUP BY dept
HAVING   AVG(cgpa) > 7.5;
```

Aggregate functions are COUNT, SUM, AVG, MIN and MAX. WHERE filters rows *before* grouping; HAVING filters groups *after* aggregation.

Subqueries and views

```
SELECT name FROM student
WHERE  cgpa > (SELECT AVG(cgpa) FROM student);

CREATE VIEW toppers AS
SELECT roll_no, name FROM student WHERE cgpa >= 9;
```

A **correlated subquery** refers to columns of the outer query and is re-evaluated for each outer row. EXISTS, IN, ANY and ALL work with subqueries. A **view** is a virtual table defined by a query; it simplifies complex queries and restricts what users can see.

Window functions

```
SELECT name, dept, cgpa,
       RANK() OVER (PARTITION BY dept ORDER BY cgpa DESC) AS dept_rank
FROM   student;
```

Window functions such as ROW_NUMBER, RANK, LAG and running SUM compute values across related rows without collapsing them into groups. Other features include common table expressions (WITH clauses), recursive queries, stored procedures, functions and triggers.

11. Functional Dependencies

A **functional dependency (FD)**, written X → Y, means that the value of attribute set X uniquely determines the value of attribute set Y. In other words, any two tuples that agree on X must also agree on Y. For example, in a STUDENT table, roll_no → name, because each roll number belongs to exactly one student.

Functional dependencies capture the meaning of data and are the main tool for judging whether a relational design is good. They are the basis of keys and of normalisation.

Types of functional dependencies

- **Trivial FD** - X → Y where Y is a subset of X, e.g. {roll_no, name} → name.
- **Non-trivial FD** - Y is not a subset of X.
- **Partial dependency** - a non-key attribute depends on only part of a composite key.
- **Transitive dependency** - X → Y and Y → Z, so Z depends on X indirectly through Y.
- **Multivalued dependency** - X →→ Y, where X determines a set of Y values independently of other attributes.

Armstrong's axioms

Armstrong's axioms are a sound and complete set of inference rules for FDs:

- **Reflexivity**: if Y □ X, then X → Y.
- **Augmentation**: if X → Y, then XZ → YZ.
- **Transitivity**: if X → Y and Y → Z, then X → Z.

Useful derived rules include **union** (X → Y and X → Z imply X → YZ), **decomposition** (X → YZ implies X → Y and X → Z) and **pseudo-transitivity**.

Attribute closure

The **closure** X^+ of an attribute set X is the set of all attributes functionally determined by X. It is computed by starting with X and repeatedly adding the right-hand side of any FD whose left-hand side is already contained in the result. If X^+ contains all attributes of the relation, X is a super key; if X is also minimal, it is a candidate key.

```
R(A, B, C, D, E)   FDs: A -> B, B -> C, CD -> E
{A, D}+ = {A, D} -> add B -> add C -> add E = {A, B, C, D, E}
So AD is a candidate key.
```

A **canonical (minimal) cover** is an equivalent set of FDs with no redundant dependencies and no extraneous attributes, and is used when designing decompositions.

12. Normalization

Normalisation is the process of organising relations to reduce redundancy and avoid **update anomalies**. Poorly designed tables suffer from *insertion anomalies* (cannot add a department until it has a student), *deletion anomalies* (deleting the last student loses the department information) and *update anomalies* (changing a department's building must be repeated in many rows).

Normal forms

- **First Normal Form (1NF)** - every attribute holds atomic values; there are no repeating groups or multivalued cells.
- **Second Normal Form (2NF)** - in 1NF and no non-prime attribute is partially dependent on any candidate key.
- **Third Normal Form (3NF)** - in 2NF and no non-prime attribute is transitively dependent on a candidate key. Equivalently, for every FD X → A, either X is a super key or A is a prime attribute.
- **Boyce-Codd Normal Form (BCNF)** - for every non-trivial FD X → Y, X is a super key. BCNF is stricter than 3NF.
- **Fourth Normal Form (4NF)** - in BCNF and has no non-trivial multivalued dependencies.
- **Fifth Normal Form (5NF)** - every join dependency is implied by the candidate keys.

Worked example

```
ENROLL(roll_no, course_id, student_name, course_title, instructor, grade)
FDs: roll_no -> student_name
     course_id -> course_title, instructor
     roll_no, course_id -> grade
```

The key is (roll_no, course_id). student_name depends only on roll_no and course_title only on course_id: these are partial dependencies, so the table is not in 2NF. Decomposing gives:

```
STUDENT(roll_no, student_name)
COURSE(course_id, course_title, instructor)
ENROLL(roll_no, course_id, grade)
```

Each resulting table is now in BCNF.

Properties of a good decomposition

A decomposition should be **lossless-join**, meaning the original relation can be recovered exactly by joining the pieces, and ideally **dependency-preserving**, meaning every FD can be checked within a single table. Decomposition into 3NF can always achieve both; BCNF guarantees losslessness but may lose some dependencies. In practice, designers sometimes **denormalise** deliberately to improve read performance in reporting systems.

13. Transactions and ACID Properties

A **transaction** is a logical unit of work that accesses and possibly updates data. A classic example is transferring money from account A to account B: read A, subtract the amount, write A, read B, add the amount, write B. Either all of these steps must happen, or none of them.

ACID properties

- **Atomicity** - all operations of a transaction are reflected in the database, or none are. If a failure occurs midway, partial changes are undone.
- **Consistency** - a transaction executed alone takes the database from one consistent state to another, preserving all integrity constraints (e.g. the total money in A and B is unchanged).
- **Isolation** - although transactions run concurrently, each appears to execute alone; intermediate results are hidden from other transactions.
- **Durability** - once a transaction commits, its changes survive any subsequent system failure.

Atomicity and durability are ensured by the **recovery manager**, isolation by the **concurrency-control manager**, and consistency is a shared responsibility of the application programmer and the DBMS's integrity constraints.

Transaction states

State	Meaning
Active	The transaction is executing
Partially committed	The final statement has executed, but changes may still be in memory
Committed	All changes are permanently recorded
Failed	Normal execution can no longer proceed
Aborted	Changes have been rolled back; the transaction may be restarted or killed

Transactions in SQL

```
BEGIN;
UPDATE account SET balance = balance - 500 WHERE acc_no = 'A';
UPDATE account SET balance = balance + 500 WHERE acc_no = 'B';
COMMIT;      -- or ROLLBACK; to undo everything
```

Schedules and serialisability

A **schedule** is the order in which operations of concurrent transactions are executed. A **serial schedule** runs transactions one after another and is always correct. A concurrent schedule is **serialisable** if its effect is equivalent to some serial schedule. **Conflict serialisability** is tested by drawing a precedence graph: the schedule is conflict-serialisable if and only if the graph has no cycle. Schedules should also be **recoverable** and preferably **cascadeless**, so that one abort does not force many others.

14. Concurrency Control

When several transactions run at the same time, uncontrolled interleaving can corrupt data. **Concurrency control** ensures that concurrent execution produces correct, serialisable results while still allowing as much parallelism as possible.

Concurrency problems

- **Lost update** - two transactions read the same value and both write, so one update overwrites the other.
- **Dirty read** - a transaction reads data written by another transaction that later aborts.
- **Non-repeatable read** - a transaction reads the same row twice and gets different values.
- **Phantom read** - a repeated query returns new rows inserted by another transaction.

Lock-based protocols

Transactions acquire a **shared (S) lock** to read and an **exclusive (X) lock** to write. Many transactions may hold S locks on an item, but an X lock is incompatible with every other lock. In **Two-Phase Locking (2PL)** a transaction has a *growing phase*, in which it acquires locks, followed by a *shrinking phase*, in which it releases them; it may not acquire a lock after releasing one. 2PL guarantees conflict serialisability. **Strict 2PL** holds all exclusive locks until commit, preventing cascading aborts.

Deadlocks

A **deadlock** occurs when transactions wait for each other in a cycle. DBMSs handle it by **prevention** (wait-die and wound-wait schemes using timestamps), **detection** (building a wait-for graph and aborting a victim when a cycle appears) or **timeouts**.

Other techniques

- **Timestamp ordering** - each transaction gets a timestamp, and conflicting operations must execute in timestamp order.
- **Optimistic (validation-based) control** - transactions run without locks and are validated at commit time.
- **Multi-Version Concurrency Control (MVCC)** - writers create new versions, so readers see a consistent snapshot and never block writers. Used by PostgreSQL, Oracle and MySQL InnoDB.

SQL isolation levels

Isolation level	Dirty read	Non-repeatable read	Phantom
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

15. Recovery System

Computer systems can fail at any time. The **recovery system** restores the database to a consistent state after a failure, guaranteeing the atomicity and durability of transactions.

Types of failures

- **Transaction failure** - a logical error (invalid input, constraint violation) or a system error such as deadlock forces a transaction to abort.
- **System crash** - a power failure or software bug loses the contents of main memory, while disk storage survives.
- **Disk failure** - a head crash or media failure destroys data on disk; recovery requires backups or replicas.

Log-based recovery

The DBMS keeps a **log** on stable storage recording every update: <T, X, old value, new value>, along with <T start>, <T commit> and <T abort> records. The **Write-Ahead Logging (WAL)** rule states that a log record must be written to stable storage *before* the corresponding data page is written to disk.

- **Undo** restores old values for transactions that did not commit.
- **Redo** reapplies new values for transactions that committed but whose changes may not have reached disk.
- With **deferred modification**, changes are written only after commit, so only redo is needed. With **immediate modification**, both undo and redo may be needed.

Checkpoints

Scanning the entire log after a crash would be slow. Periodically the system takes a **checkpoint**: it flushes log records and modified buffer pages to disk and writes a <checkpoint L> record listing active transactions. Recovery then only needs to examine the log from the last checkpoint onward.

ARIES

Most commercial systems use **ARIES** (Algorithm for Recovery and Isolation Exploiting Semantics). Recovery proceeds in three passes: **analysis** determines which transactions were active and which pages were dirty; **redo** repeats history to bring the database to its exact state at the crash; and **undo** rolls back incomplete transactions, writing compensation log records so that recovery is itself safe to repeat.

Other techniques

Shadow paging keeps two page tables and switches atomically on commit, avoiding the need for undo, but causes fragmentation. For disk failures, systems use regular full and incremental **backups**, archived logs for point-in-time recovery, RAID, and **replication** to standby servers in other locations.

16. Indexing and Hashing

Queries often need only a small fraction of the rows in a table. Scanning every block is slow, so DBMSs build **indexes**: auxiliary structures that map search-key values to the location of records, much like the index at the back of a book.

Types of indexes

- **Primary (clustering) index** - built on the attribute by which the file is physically ordered. There can be only one.
- **Secondary (non-clustering) index** - built on another attribute; the file is not sorted by it.
- **Dense index** - has an entry for every search-key value.
- **Sparse index** - has entries for only some values, typically one per block; only possible for clustered files.
- **Multilevel index** - an index on the index, used when the index itself becomes too large for memory.

B+ trees

The **B+ tree** is the default index structure in nearly every relational DBMS. It is a balanced tree in which all leaves are at the same depth; internal nodes guide the search, and leaf nodes store keys with record pointers and are linked together for efficient range scans. Each node holds between ⌈n/2⌉ and n children, which keeps the tree shallow: a tree of height 3 or 4 can index millions of rows. Search, insertion and deletion all take O(log n) time, and the tree rebalances itself by splitting and merging nodes.

Hashing

Hash indexes apply a hash function to the search key to compute the bucket where the record lives. Lookups for equality conditions take near-constant time, but hashing cannot support range queries. **Static hashing** uses a fixed number of buckets and suffers from overflow chains as data grows. **Dynamic hashing** techniques such as **extendible hashing** and **linear hashing** grow the number of buckets gradually.

Creating an index in SQL

```
CREATE INDEX idx_student_dept ON student(dept);
CREATE UNIQUE INDEX idx_student_email ON student(email);
CREATE INDEX idx_enroll ON enrollment(roll_no, course_id);  -- composite
```

Trade-offs

Indexes speed up reads but slow down inserts, updates and deletes, because every index must be maintained, and they consume storage. Good candidates are columns used frequently in WHERE clauses, joins and ORDER BY. Other specialised structures include **bitmap indexes** for low-cardinality columns, **GiST/R-trees** for spatial data and **inverted indexes** for full-text search.

17. Query Processing and Optimization

When a user submits an SQL query, the DBMS must decide how to execute it efficiently. Because SQL is declarative, there may be many different execution plans that produce the same result, with costs that differ by orders of magnitude.

Steps in query processing

- **Parsing and translation** - the query is checked for syntax and validated against the catalog, then translated into an internal relational-algebra form.
- **Optimisation** - the optimiser generates alternative equivalent plans and estimates the cost of each, choosing the cheapest.
- **Evaluation** - the execution engine runs the chosen plan and returns the results.

Evaluating operations

Selection may use a linear scan, binary search or an index. **Joins** are the most expensive operations and have several algorithms: **nested-loop join** (simple, works for any condition), **block nested-loop join**, **indexed nested-loop join**, **sort-merge join** (efficient when inputs are sorted) and **hash join** (very efficient for large equi-joins). **Sorting** of data larger than memory uses external merge sort.

Heuristic optimisation

Optimisers apply equivalence rules of relational algebra to transform queries. Common heuristics are:

- Perform selections as early as possible to reduce the size of intermediate results.
- Perform projections early to drop unneeded columns.
- Replace a Cartesian product followed by a selection with a join.
- Execute the most restrictive joins first.

Cost-based optimisation

Cost-based optimisers use **statistics** stored in the catalog, such as the number of rows, number of distinct values and histograms of value distributions, to estimate the size of intermediate results and the number of disk I/Os. Dynamic programming is used to search join orders. Results are pipelined between operators where possible instead of being written to temporary tables.

Viewing a plan

```
EXPLAIN ANALYZE
SELECT s.name, c.title
FROM student s JOIN enrollment e ON s.roll_no = e.roll_no
JOIN course c ON c.course_id = e.course_id
WHERE s.dept = 'CSE';
```

Reading execution plans helps developers spot full table scans, missing indexes and poor join orders.

18. Database Security and Authorization

Databases often hold the most valuable information an organisation owns: financial records, personal details, medical histories and trade secrets. **Database security** protects this data against unauthorised disclosure, alteration and destruction.

Threats

- **Loss of confidentiality** - sensitive data is disclosed to unauthorised parties.
- **Loss of integrity** - data is improperly modified.
- **Loss of availability** - legitimate users cannot access data when needed.
- **SQL injection** - an attacker inserts malicious SQL through unvalidated application input.

Access control

Discretionary Access Control (DAC) lets owners grant privileges to other users with SQL's GRANT and REVOKE commands. **Mandatory Access Control (MAC)** assigns security labels such as Top Secret or Confidential to data and users, and the system enforces rules like 'no read up, no write down'. **Role-Based Access Control (RBAC)** groups privileges into roles, such as clerk or manager, and assigns roles to users, which simplifies administration.

```
CREATE ROLE analyst;
GRANT SELECT ON sales TO analyst;
GRANT analyst TO priya;
GRANT UPDATE (salary) ON employee TO hr_manager WITH GRANT OPTION;
REVOKE SELECT ON sales FROM analyst;
```

Preventing SQL injection

Never build SQL by concatenating user input. Instead use **parameterised queries** (prepared statements), which send the query text and data separately:

```
cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
```

Other security measures

- **Authentication** with strong passwords, multi-factor authentication or certificates.
- **Views** that expose only permitted rows and columns.
- **Encryption** of data at rest (transparent data encryption) and in transit (TLS).
- **Auditing** that logs who accessed or changed which data and when.
- **Data masking** and anonymisation for test environments and analytics.
- Compliance with regulations such as GDPR and HIPAA, and following the principle of least privilege.

19. NoSQL and Distributed Databases

The growth of web-scale applications, big data and cloud computing exposed limits of traditional single-server relational databases. **Distributed databases** spread data across many machines, and **NoSQL** ('not only SQL') systems relax some relational features to gain scalability, availability and flexible schemas.

Distributed database concepts

- **Fragmentation (sharding)** - a table is split horizontally (by rows) or vertically (by columns) across sites.
- **Replication** - copies of data are kept on several nodes for availability and faster reads.
- **Distributed transactions** - use protocols such as **Two-Phase Commit (2PC)** so that all sites commit or all abort.
- **Transparency** - users should not need to know where data is located or how it is replicated.

The CAP theorem

Eric Brewer's **CAP theorem** states that a distributed system cannot simultaneously guarantee **Consistency** (every read sees the latest write), **Availability** (every request receives a response) and **Partition tolerance** (the system continues despite network splits). Since partitions are unavoidable, systems choose between consistency and availability during a partition. Many NoSQL systems favour **BASE** semantics: Basically Available, Soft state, Eventually consistent.

Types of NoSQL databases

Type	Data structure	Examples	Typical use
Key-value	Key mapped to an opaque value	Redis, DynamoDB	Caching, sessions
Document	JSON/BSON documents	MongoDB, Couchbase	Content, catalogues
Column-family	Rows with dynamic column groups	Cassandra, HBase	Time series, logs
Graph	Nodes and edges	Neo4j, Amazon Neptune	Social networks, fraud

Choosing between SQL and NoSQL

Relational databases remain the best choice when data is highly structured, relationships are complex and strong consistency with ACID transactions is essential, such as in banking. NoSQL databases shine when data volume is enormous, schemas change frequently, or very high write throughput and horizontal scaling are required. Modern **NewSQL** systems such as Google Spanner and CockroachDB aim to combine SQL and ACID guarantees with distributed scalability.

20. Summary and Future Trends

This document has traced the main ideas of database management systems, from the motivation for moving beyond file systems to the internal machinery that makes modern databases fast, reliable and secure.

Key takeaways

- A DBMS provides centralised, consistent, secure and concurrent access to data, with data independence through the three-schema architecture.
- Conceptual design uses the ER model, which is then mapped to relations in the relational model.
- Keys and integrity constraints preserve the correctness of data.
- Relational algebra underpins SQL, the universal language for querying and manipulating relational data.
- Functional dependencies and normalisation remove redundancy and update anomalies.
- Transactions with ACID properties, concurrency control and recovery keep data correct despite concurrency and failures.
- Indexes and query optimisation make queries fast, and security mechanisms protect data from misuse.
- Distributed and NoSQL systems trade some guarantees for scale and flexibility.

Emerging trends

- **Cloud and serverless databases** such as Amazon Aurora, Azure SQL and Google BigQuery, which scale automatically and are billed by usage.
- **Vector databases** such as Pinecone, Milvus, Chroma, FAISS and pgvector, which store embeddings and support similarity search for AI and retrieval-augmented generation (RAG) applications.
- **HTAP systems** that handle transactional and analytical workloads on the same data.
- **Data lakes and lakehouses** that combine cheap object storage with table formats like Delta Lake and Apache Iceberg.
- **Autonomous databases** that use machine learning to tune indexes, memory and query plans automatically.
- **Natural-language interfaces** in which large language models translate questions into SQL.

Further reading

- Silberschatz, Korth and Sudarshan - *Database System Concepts*.
- Elmasri and Navathe - *Fundamentals of Database Systems*.
- Ramakrishnan and Gehrke - *Database Management Systems*.
- Kleppmann - *Designing Data-Intensive Applications*.

Databases remain one of the foundational technologies of computing. A solid grasp of these fundamentals is valuable for any software engineer, data scientist or AI practitioner.